

Loka Framework Index

Version: 2.2 **Date:** 10 May 2026 **Purpose:** The front door for the Loka framework family. Pick the right Loka module for your situation in under 60 seconds.

What Is Loka?

Loka is a family of operating frameworks for making AI work more reliably. Each module gives the AI a clear operating constitution so it can produce structured, safer, more useful work instead of vague output, spaghetti code, exposed secrets, or unsupported completion claims.

Loka does not make AI perfect. It improves discipline by enforcing context, scope, structure, constraints, evidence, and honest readiness.

Decision Tree — Which Loka Module Do I Use?

Answer the first question that applies. Stop there.

1. Is your request vague and you need to shape it before execution?

Use **Loka_ACE**.

It is the Adaptive Context Engine. Use it to turn unclear intent into a clean AI operating context for research, strategy, content, briefs, workflows, assistant design, audits, education, and decision support.

Use **Loka_ACE Lite** for daily paste use. Use **Loka_ACE Full** for onboarding, teaching, examples, and templates.

2. Are you writing prompts, planning an app/SaaS, or auditing without live coding tools?

Use **LokaV10**.

It is the Secure App Builder & Prompt Engineering Framework. Use it for prompt

architecture, SaaS planning, implementation planning, audits, and safe app thinking when the AI does not have repo/shell execution capability.

3. Are you using a CLI/IDE coding agent with repo, file, shell, or tool access?

Use **LokaV11**.

It is the Agentic Build Operating System. Use it for Claude Code, Cursor, Aider, Codex CLI, Gemini CLI, and similar tools that can inspect files, edit code, run commands, or work inside a repo.

New in v11.3: Dependency Verification Protocol and Surgical Fix Mode.

4. Do you need a supervised 1–3 hour coding session with checkpoints?

Use **LokaV11 + LokaV12**.

LokaV12 loads on top of LokaV11. It adds autonomy levels, pre-flight checks, cost/time ceilings, checkpointing, kill switches, rollback discipline, and end-of-session reports.

5. Are you building inside a prompt-driven visual builder or an agentic visual IDE?

Use **LokaV13**.

It is the Agentic Visual Builder OS. Use it for Google AI Studio Build mode, Google Antigravity IDE, Lovable, Bolt, v0, Replit Agent, and similar environments where the primary interaction is prompt-to-app rather than manual file editing.

What V13 now covers (v13.2): Both AI Studio Build mode (browser, agent-managed shell) and Antigravity IDE (local install, full direct shell access with tiered terminal policy). V13 governs terminal permission tiers, browser sub-agent verification, checkpoint approval discipline, durable project memory via `.gemini/antigravity/brain/`, and Export Prep handoff to LokaV11.

When to switch from V13 to V11: When you export your project from AI Studio or Antigravity into a standalone IDE with full repo control — stop using V13 and switch to V11 at that point.

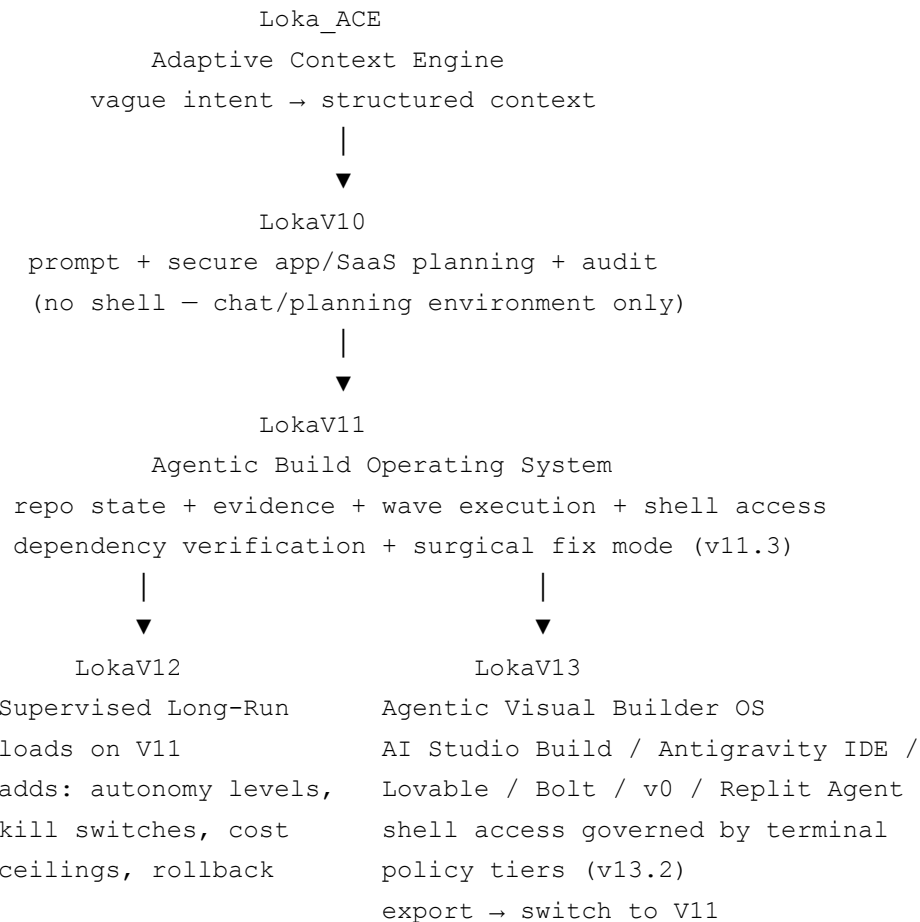
6. Still not sure?

Use **Loka_ACE first** to shape the task. Then hand off to LokaV10, V11, V12, or V13 if the task becomes specialized.

Quick Selection Table

Situation	Use
Vague task, unclear goal, messy idea	Loka_ACE
Research, strategy, content, assistant design, workflow planning	Loka_ACE
Prompt engineering, SaaS planning, technical audit, implementation plan	LokaV10
Chat-only environment without tools	LokaV10
Claude Code, Cursor, Aider, Codex CLI, Gemini CLI, repo/file/shell agent	LokaV11
Targeted bug fix or scoped edit in existing repo	LokaV11 — Surgical Fix Mode
Supervised long-run build session	LokaV11 + LokaV12
Google AI Studio Build mode (browser)	LokaV13
Google Antigravity IDE (local install)	LokaV13
Lovable, Bolt, v0, Replit Agent, visual prompt builders	LokaV13
Messy AI Studio / Antigravity project rescue	LokaV13 — Rescue Mode
Exporting from AI Studio / Antigravity to a real IDE	LokaV13 Export Prep → then switch to LokaV11
Existing messy IDE/repo project audit	LokaV11 Audit or LokaV10 Audit depending on tool access

Visual Map



Lite vs Full

Form	Best For
Lite	Daily use, system prompts, mobile use, lower context cost, quick sessions
Full	Onboarding, teaching, reference, templates, complex systems, documentation

Lite versions contain the enforced core. Full versions explain the reasoning, examples, and templates.

Loading Rules

Loka_ACE

Use alone for general context design. If the task becomes technical, hand off to the correct

specialized framework.

LokaV10

Use alone for prompt architecture, secure app planning, audits, and implementation planning where the AI does not have repo/shell access.

LokaV11

Use alone for coding agents with repo/file/shell/tool access.

LokaV12

Always load **on top of LokaV11**. Do not use V12 by itself.

LokaV13

Use alone for visual prompt-driven builders and agentic visual IDEs (AI Studio Build mode, Antigravity IDE, Lovable, Bolt, v0, Replit Agent).

When the project is exported to a standalone IDE with full repo control: **stop using V13 and switch to V11**. Do not run V13 and V11 simultaneously on the same project.

Common Mistakes To Avoid

1. **Loading multiple Loka versions at once.** Pick the right one. Only V12 is designed to stack, and only on V11.
2. **Using ACE when execution is already clear.** If you already know you are building in AI Studio or Antigravity, go straight to V13.
3. **Using V10 when the agent can edit files and run commands.** Use V11 instead.
4. **Continuing to use V13 after exporting to a real IDE.** Switch to V11 the moment you export. V13 governs the visual builder environment; V11 governs the repo.
5. **Skipping state files in real projects.** V11 and V13 both depend on durable project memory. Without state files, the AI builds on stale context.
6. **Trusting preview or browser sub-agent visual check as full verification.** Preview proves the app renders. Browser sub-agent confirms visual layout. Neither proves DB writes, auth, payments, or backend logic actually work.
7. **Treating Lite as a downgrade.** Lite is the daily-use core. Full is the reference version.

8. **Expecting persona to replace process.** Roles help, but execution steps, constraints, and evaluation criteria do most of the work.
 9. **Setting Antigravity terminal policy to Always Proceed on real projects.** Start at Request Review. Escalate to Auto only after trust is established. Never use Always Proceed with real data or credentials.
 10. **Assuming dependency versions are current without verification.** Use the Dependency Verification Protocol (V11 Section 2.1 / V13 Section 4) before building on any version declared from training knowledge alone.
-

File Manifest

LOKA_INDEX.md	← front door and decision tree (v2.2)
LOKA_HOW_TO_USE.md	← practical usage guide (v2.1)
LOKA_FAILURES.md	← real AI build failure modes (v1.1 – 20 patterns)
Loka_ACE_Full.md	← Adaptive Context Engine – Full (v1.3)
Loka_ACE_Lite.md	← Adaptive Context Engine – 1-page core (v1.1)
Loka_ACE_Example_Pack.md	← reusable professional examples (v1.1)
LokaV10.md	← Secure App Builder & Prompt Engineering – Full
LokaV10_Lite.md	← Secure App Builder – 1-page core
LokaV11.md	← Agentic Build OS – Full (v11.3)
LokaV11_Lite.md	← Agentic Build OS – 1-page core (v11.3)
LokaV12.md	← Supervised Long-Run – Full
LokaV12_Lite.md	← Supervised Long-Run – 1-page core
LokaV13.md	← Agentic Visual Builder OS – Full (v13.2)
LokaV13_Lite.md	← Agentic Visual Builder OS – 1-page core (v13.2)

One-Line Summary

Use **Loka_ACE** to shape the task. Use **LokaV10** to plan or audit without shell access. Use **LokaV11** to build with coding agents. Use **LokaV12** for supervised long-runs. Use **LokaV13** for visual builders and agentic visual IDEs — then switch to V11 after export.

End Loka Framework Index v2.2.

